

Strattum — Benchmark do Lakehouse Aberto

🇬🇧 English version: [BENCHMARK-LAKEHOUSE.en.md](#)

De 2 bancos operacionais a um grafo de conhecimento, em minutos, num notebook. 3 milhões de registros (PostgreSQL + MongoDB) → lakehouse aberto (DuckLake + Parquet/S3) → camada *clean* modelada → grafo de entidades — medido ponta a ponta.

Este documento reporta um benchmark **reprodutível** da plataforma Strattum rodando **inteira num único MacBook (8 GB de RAM)**. O objetivo é responder, com números, a uma pergunta simples: *quão rápido e quão barato qualquer empresa consegue transformar os próprios dados operacionais em algo útil?*

TL;DR

O que	Volume	Tempo	Throughput	Pico de RAM
Ingestão PostgreSQL → lake	2.000.000 linhas	36,9 s	~54.200 linhas/s	272 MB
Ingestão MongoDB → lake	1.000.500 docs	25,0 s	~40.500 docs/s	272 MB
Transformação (dbt/DuckLake)	3.000.500 linhas	3,8 s	~795.000 linhas/s	382 MB
Sync incremental (+150k)	150.000 linhas	3,5 s	delta-only, 0 duplicatas	267 MB
Grafo de conhecimento	290.756 nós · 300.000 arestas	126 s ¹	~4.680 elem/s	250 MB (worker) · 302 MB (FalkorDB)

¹ Grafo numa **fatia representativa** (100k contratos + entidades ligadas); build otimizado de 362 s → 126 s durante o benchmark (ver §4). Ingestão e *clean* rodaram no volume **cheio**.

3 milhões de linhas ingeridas + modeladas em ~66 segundos, com pico de ~380 MB de RAM por processo — em hardware de mesa, sobre formatos 100% abertos (Parquet + catálogo DuckLake), sem data warehouse proprietário e sem cluster.

O cenário

Uma empresa média tem os dados espalhados: um **PostgreSQL** com o cadastro de pessoas, um **MongoDB** com os contratos/processos, planilhas, um ERP. Ninguém "faz nada com data" porque juntar tudo isso normalmente exige um projeto de meses, um data warehouse caro e um time dedicado.

O benchmark simula exatamente esse caso com dados sintéticos realistas:

- **PostgreSQL** — **users** : 2.000.000 de usuários (id, nome, e-mail, CPF, cidade, timestamps).

- **MongoDB — 2 coleções:**
 - `contracts` : 1.000.000 de contratos, cada um com **duas partes** (`processante` e `processado` , ambos referenciando um usuário) + produto, status e valor;
 - `products` : 500 produtos (id → nome/categoria).
 - **Incremento:** +100.000 usuários e +50.000 contratos, para medir o custo de um sync do dia seguinte.
- O alvo: uma camada *clean* com **uma tabela de usuários** e **uma tabela de contratos** (já com o `JOIN` das duas coleções do Mongo, trazendo o nome do produto), e um **grafo** onde cada contrato conecta suas duas partes e seu produto.

Setup (deliberadamente modesto)

Item	Valor
Máquina	MacBook, Apple Silicon, 8 vCPU, 8 GB RAM
Docker (VM Linux)	3,8 GB para todos os containers
Fontes	PostgreSQL 16, MongoDB 7 (containers)
Lakehouse	DuckLake (catálogo em Postgres) + Parquet em MinIO (S3)
Transformação	dbt rodando DuckDB/DuckLake nativo
Grafo	FalkorDB (grafo de entidades)
Orquestração	Prefect

Nada aqui é "hardware de benchmark". É o que qualquer dev tem na mesa. O ponto é justamente esse: **a arquitetura aberta escala para baixo** — roda no notebook hoje e no cluster amanhã, sem trocar de formato.

Resultados por estágio

1) Ingestão (bootstrap — carga cheia)

Cada conector **faz streaming** da fonte (cursor server-side no Postgres, cursor em lote no Mongo) e grava Parquet no lake em micro-batches. Por isso o **pico de memória é constante (~272 MB)** **independentemente do volume** — o dataset inteiro nunca é carregado na RAM.

Fonte	Linhas	Tempo	Throughput	Pico RAM	Parquet gerado
PostgreSQL <code>users</code>	2.000.000	36,9 s	54.200 I/s	272 MB	83 MB
MongoDB <code>contracts</code>	1.000.000	24,7 s	40.500 I/s	272 MB	85 MB
MongoDB <code>products</code>	500	0,2 s	—	—	36 KB
Total	3.000.500	~62 s	~48.500 I/s	272 MB	~168 MB

3 milhões de linhas viram ~168 MB de Parquet colunar comprimido — a base já fica pronta para consulta analítica direta, sem cópia para um warehouse.

2) Transformação — camada *clean* (dbt sobre DuckLake)

O dbt lê o `raw` e materializa três tabelas *clean* no próprio lake. A `clean.contracts` faz o `JOIN` das duas coleções do Mongo (contratos x produtos), normaliza tipos e traz o nome do produto para cada contrato.

Modelo	Linhas	Tempo
<code>clean.products</code>	500	0,40 s
<code>clean.users</code>	2.000.000	3,23 s
<code>clean.contracts</code> (join)	1.000.000	3,44 s
Total	3.000.500	3,8 s

Resultado: **0 contratos sem produto** (join íntegro), ~795 mil linhas/s. DuckDB sobre Parquet transforma milhões de linhas em segundos — sem mover dado para fora do lake.

3) Sync incremental (o "dia seguinte")

Depois de +100k usuários e +50k contratos nas fontes, o mesmo pipeline roda de novo. O cursor incremental lê **apenas o delta** (`updated_at > watermark`) e faz *merge* por chave primária — nada é reprocessado.

Fonte	Delta	Tempo	vs. bootstrap
PostgreSQL	100.000	2,2 s	~17x mais rápido
MongoDB	50.000	1,3 s	~19x mais rápido

Verificado: `raw` foi para **exatamente** 2.100.000 / 1.050.000 linhas, **0 duplicatas** (o *merge* delete-then-insert por PK dedupe corretamente) e o watermark avançou. O custo do sync acompanha o **tamanho da mudança**, não o tamanho da base — é isso que torna a plataforma barata em regime permanente.

4) Grafo de conhecimento (FalkorDB)

A ontologia mapeia a camada *clean* para um grafo: `User`, `Contract` e `Product`, com as arestas **PROCESSANTE** e **PROCESSADO** (contrato → usuário) e **SOBRE** (contrato → produto). A resolução de entidade é determinística (mesmo id → mesmo nó), então as duas partes de cada contrato apontam para os nós de usuário corretos.

Por que uma fatia? Materializar o grafo **completo** (2,1M usuários + 1,05M contratos ⇒ ~3,15M nós + ~3,15M arestas) exigiria **~3,3 GB só no FalkorDB** (extrapolado da medição real, ~510 bytes/elemento) — não cabe na VM Docker de 3,8 GB do notebook, que ainda divide RAM com Postgres/Mongo/MinIO. Então o grafo foi materializado sobre uma **fatia representativa** — 100k contratos + os 190.256 usuários referenciados + 500 produtos ≈ **291k nós, 300k arestas** — medido, e a taxa projetada pro volume total. Ingestão e *clean* rodaram sempre no volume **cheio** (2M/1M).

O build passou por uma **otimização durante o próprio benchmark**. Estava fazendo **1 MERGE por linha** (um round-trip ao FalkorDB por nó e por aresta); reescrevi o `execute_batch` do memory-worker pra **agrupar cada lote num único** `UNWIND $rows AS row ...` (1 round-trip por lote de ~500):

Métrica	Antes (1 MERGE/linha)	Depois (UNWIND)	Projeção grafo cheio
Nós	290.756	290.756	~3,15M
Arestas	300.000	300.000	~3,15M
Tempo de build	362 s	126 s	~22 min
Taxa	~1.630 elem/s	~4.680 elem/s	—
Pico RAM (worker)	255 MB	250 MB	—
RAM FalkorDB	302 MB	302 MB	~3,3 GB

2,87x mais rápido, resultado idêntico, **0 fallbacks**. O ganho local é "só" ~3x porque na mesma máquina o round-trip é barato — o custo dominante virou o próprio `MERGE` no FalkorDB + a leitura do lake. **Num FalkorDB remoto (produção, outro host), onde a latência de rede domina, o UNWIND rende bem mais.** Próximo passo pra carga inicial de grafos grandes: `falkordb-bulk-loader` (CSV → grafo, milhões/s).

Qualidade: 100% das arestas resolvidas — PROCESSANTE 100.000/100.000, PROCESSADO 100.000/100.000, SOBRE 100.000/100.000, **0 órfãs**. A resolução de entidade determinística garante que as duas partes de cada contrato caem nos nós de usuário certos.

Sub-grafo real (um contrato e seu "digital twin") direto do FalkorDB:

```
MATCH (c:Contract)-[:PROCESSANTE]->(a:User),
      (c)-[:PROCESSADO]->(b:User),
      (c)-[:SOBRE]->(p:Product)
RETURN c.contract_id, a.name, b.name, p.product_name, c.status, c.value LIMIT 2
```

contract_id	processante	processado	produto	status	valor
95152	Usuário 400774	Usuário 5093	Produto 54	encerrado	42.933,20
88258	Usuário 1968098	Usuário 8993	Produto 485	encerrado	126.069,85

Digital twin — um contrato e suas ligações

Sub-grafo real do FalkorDB: 1 contrato (Mongo) conecta suas duas partes (Postgres) e o produto (Mongo)



(Achados do benchmark no viz do grafo, já corrigidos: **(a)** o first-paint pedia 5.000 nós e estourava (HTTP 500) em grafos grandes — cap baixado; **(b)** o first-paint trazia só os nós de **entidade** e, num grafo hub-and-spoke (todo vínculo passa pelo **contrato**), isso pintava os usuários **soltos, sem nenhuma conexão** — "só vejo os nós". Corrigido: o primeiro paint agora vem de uma **amostra semeada por arestas** (via a query Cypher read-only), então já abre **conectado** — os "digital twins" acima aparecem de cara, sem precisar clicar. A expansão por-nó ainda não devolve todas as arestas de entrada — follow-up separado.)

A ontologia que gera esse grafo

O grafo acima não é hardcoded — sai de um **mapa declarativo** (`graph_mapping.yaml`): nós `User / Product / Contract` (cada um lendo uma tabela `clean`) + arestas `PROCESSANTE / PROCESSADO / SOBRE` ligando contrato → partes e produto. Como `postgres / mongodb` são *dynamic-schema* (o schema é do cliente), esse mapa é **escrito por cliente** — aqui, à mão, como faria o FDE. Duas formas de criar/editar, ambas gravando a **mesma** coisa, versionada:

- **Pela API** (FDE / automação): `PUT /v1/ontology` (salva) + `POST /v1/ontology/apply` (aplica) — foi assim que registrei a **Versão 4** deste benchmark.
- **Pela UI** (cliente): **Configurações** → **Ontologia** → **aba YAML** → **Editar** → **Salvar e aplicar**. O console valida o YAML e as colunas contra as tabelas `clean` reais, salva e aplica num clique, com histórico de versões pra rollback.

strattum.ai

Buscar conectores, memory, knowledge, docs...

acme-prod

Dev User

MENU

Dashboard

CAMADA DE DADOS

Conectores

Transformações

Catálogo de Dados

Lineage

IA / OPS

Memory

Knowledge

Skills

OPERAÇÕES

Health

Pipelines

Evals

Observability

ACESSO A DADOS

Chaves de API

Conexões

Uso

WORKSPACE

Meu Perfil

Tímes

Usuários

IA & DADOS

Modelos de IA

FinOps

Ontologia

Glossário

Workspace Git

Ontologia

Configure os tipos de entidade, relacionamentos e mapeamento de fontes.

Grafo

YAML

Versão 4

Atual

Editar

```
1 version: "1"
2
3 # Benchmark ontology: Users, Contracts and Products from the DuckLake clean layer.
4 # Graph node sources are BOUNDED slices (clean/graph_users, clean/graph_contracts)
5 # so the graph fits the 3.8 GB Docker VM; clean.users / clean.contracts hold the
6 # full 2M / 1M for the catalog + throughput numbers. Contract parties resolve to
7 # Users by the postgres user id (processante_id / processado_id -> user_id).
8
9 nodes:
10   - label: User
11     source: clean/graph_users
12     connector: postgres
13     id_field: user_id
14     er_fields: [email]
15     display_tier: entity
16     display_color: blue
17     properties: [user_id, name, email, cpf, city, updated_at]
18
19   - label: Product
20     source: clean/products
21     connector: mongodb
22     id_field: product_id
23     er_fields: []
24     display_tier: entity
25     display_color: green
26     properties: [product_id, product_name, product_category]
27
28   - label: Contract
29     source: clean/graph_contracts
30     connector: mongodb
31     properties: [contract_id, contract_name, contract_status]
```

Ver histórico de versões

Footprint, projeção e custos no cliente (AWS)

Quanto os dados ocupam (medido)

Onde	Tamanho	Nota
Lake — raw (Parquet/S3)	171 MB	os 3,15M registros comprimidos
Lake — clean	174 MB	tabelas materializadas
Grafo — FalkorDB (RAM)	302 MB	290k nós + 300k arestas, in-memory (~510 B/elem)
Grafo — snapshot em disco (RDB)	84 MB	persistência comprimida
(comparação) Postgres de origem	306 MB	os mesmos 2,1M users, OLTP
(comparação) Mongo de origem	170 MB	os mesmos 1,05M contratos

Dois pontos que viram argumento comercial: **(a)** o Parquet aberto (171 MB) é **mais compacto que os bancos operacionais de origem** (306 + 170 MB) — o dado continua seu, consultável por qualquer engine, sem cópia proprietária; **(b)** ingestão e clean têm pico de RAM **constante** (272–394 MB) *independente do volume* — streaming, não carregam o dataset.

O que isso aguenta num servidor de verdade

O benchmark rodou num notebook de 8 GB de propósito. Num servidor, cada estágio escala diferente — e o gargalo **não** é onde a maioria pensa.

STRATTUM — CONFIDENCIAL

pág. 6 / 14

Lake (Parquet em S3) — praticamente ilimitado, e custa centavos. A ~56 bytes/linha comprimido (medido: 3M linhas → 171 MB), o storage dos *arquivos* é a parte mais barata da conta. Preço do S3 Standard por GB·mês: **US\$ 0,023** em us-east-1 · **US\$ 0,0405** em São Paulo (sa-east-1):

Linhas	Parquet no lake	S3 us-east-1	S3 São Paulo
100 milhões	~5,6 GB	~US\$ 0,13/mês	~US\$ 0,23/mês
1 bilhão	~56 GB	~US\$ 1,30/mês	~US\$ 2,27/mês
10 bilhões	~560 GB	~US\$ 13/mês	~US\$ 23/mês

O catálogo DuckLake não adiciona custo próprio: são ~30 tabelas de metadados no **mesmo Postgres** que a plataforma já roda. Requests S3 (PUT/GET) são desprezíveis neste padrão de acesso — a ingestão escreve arquivos de micro-batch (poucos milhares de PUTs por sync, a US\$ 0,005/1k) e as leituras são scans colunares, não milhões de GETs pequenos.

Ingestão — linear no tempo, RAM constante (~48,5k linhas/s, ~272 MB fixos → nunca dá OOM): 100M linhas ≈ **~34 min**, 1B linhas ≈ **~5,7 h**. Com mais cores / conectores em paralelo / S3 mais rápido, encurta.

Clean (dbt/DuckDB) paraleliza por core e derrama pra disco — 100M linhas em ~poucos minutos num servidor comum.

Incremental — o segredo do custo baixo. O sync do dia-a-dia custa o **tamanho da mudança**, não o da base. Um cliente com 1 bilhão de linhas mas 5 milhões mudando por dia paga ~5M/dia (~2 min) — **não** 1 bilhão. Em regime permanente, a conta é sempre do delta.

Grafo (FalkorDB) — limitado por RAM, mas um servidor normal segura dezenas a centenas de milhões de entidades (a ~510 B/elemento medido, com ~40% de folga pra RAM de query):

RAM do servidor	Elementos no grafo	≈ Entidades (nós)
16 GB	~19M	~9M
32 GB	~38M	~19M
64 GB	~75M	~37M
128 GB	~150M	~75M
256 GB	~300M	~150M

O FalkorDB oficialmente vai a **bilhões de arestas** com hardware adequado, queries em milissegundos (GraphBLAS). O limite prático da **carga inicial** é a velocidade de escrita — hoje ~4.680 elem/s (com UNWIND); pra grafos grandes, o `falkordb-bulk-loader` faz milhões/s. Depois disso, o worker mantém o grafo **por delta** (barato).

Quanto custa deixar o grafo em RAM (AWS)

RAM é o único recurso que o grafo consome de verdade — e na AWS ela é comprada junto com a instância. Para grafos que ainda cabem na VM da plataforma, o custo extra é **zero**: na VM starter de 16 GB, o FalkorDB fica com uma fatia de ~4 GB (teto de cgroup do compose) → **até ~4–5M elementos sem nenhum custo adicional**. Acima disso, o grafo vai pra um **nó dedicado memory-optimized** (família `r8g`, Graviton4 — 8 GB de RAM por vCPU, o melhor US\$/GB da AWS). Preços on-demand Linux (jul/2026):

Elementos no grafo	RAM (c/ folga de query)	Instância	us-east-1	São Paulo
até ~5M	fatia da VM starter	— (compartilhada)	US\$ 0 extra	US\$ 0 extra
~19M	16 GB	r8g.large	~US\$ 86/mês	~US\$ 162/mês
~38M	32 GB	r8g.xlarge	~US\$ 172/mês	~US\$ 324/mês
~75M	64 GB	r8g.2xlarge	~US\$ 344/mês	~US\$ 647/mês
~150M	128 GB	r8g.4xlarge	~US\$ 688/mês	~US\$ 1.294/mês

Ou seja: **RAM de grafo custa ~US\$ 5,4/GB-mês em us-east-1 e ~US\$ 10/GB-mês em São Paulo** (on-demand; um savings plan de 1 ano corta ~35%). A persistência do grafo (snapshot RDB + AOF) vai pro disco EBS e é irrelevante na conta — os 590k elementos deste benchmark geraram **84 MB** de snapshot (~US\$ 0,01/mês em gp3). A regra operacional que já usamos no compose vale aqui também: o teto de RAM da instância deve ficar **acima** do `maxmemory` do FalkorDB, para o fork do BGSAVE/AOF-rewrite ter folga.

A VM da plataforma (o resto da conta)

O restante da stack (ingestão, dbt, APIs, Prefect, Postgres, Qdrant) roda na VM starter — `t4g.xlarge` (4 vCPU / 16 GB, Graviton2, ADR-005): **~US\$ 98/mês** em us-east-1 · **~US\$ 157/mês** em São Paulo, mais o disco EBS gp3 (US\$ 0,08 · US\$ 0,152 por GB-mês — 100 GB ≈ US\$ 8 · US\$ 15). Ingestão e clean são **streaming com RAM constante** (§1-2), então a mesma VM serve de 3M a centenas de milhões de linhas — o tempo cresce, a máquina não.

Exemplo — cliente médio (2M clientes + 8M contratos + ~100k mudanças/dia), conta AWS completa em São Paulo:

- **Lake (arquivos):** ~600 MB de Parquet → **~US\$ 0,03/mês** de S3.
- **Ingestão inicial** ~3-4 min · **clean** ~1 min (na própria VM).
- **Grafo:** ~10M nós + ~24M arestas ≈ 34M elementos → 48 GB c/ folga → nó dedicado `r8g.2xlarge` (64 GB) → **~US\$ 647/mês** (us-east-1: ~US\$ 344); carga inicial via bulk-loader em minutos.
- **VM da plataforma:** `t4g.xlarge` + 100 GB gp3 → **~US\$ 172/mês** (us-east-1: ~US\$ 106).
- **Sync diário:** ~100k linhas → **segundos** (o delta é a conta permanente, e é ínfima).

Total infra AWS: ~US\$ 820/mês em São Paulo (~US\$ 450 em us-east-1), on-demand — e o item dominante é a RAM do grafo, que só cresce se o *grafo* crescer. Sem warehouse proprietário, sem cluster, tudo BYOC na conta do cliente.

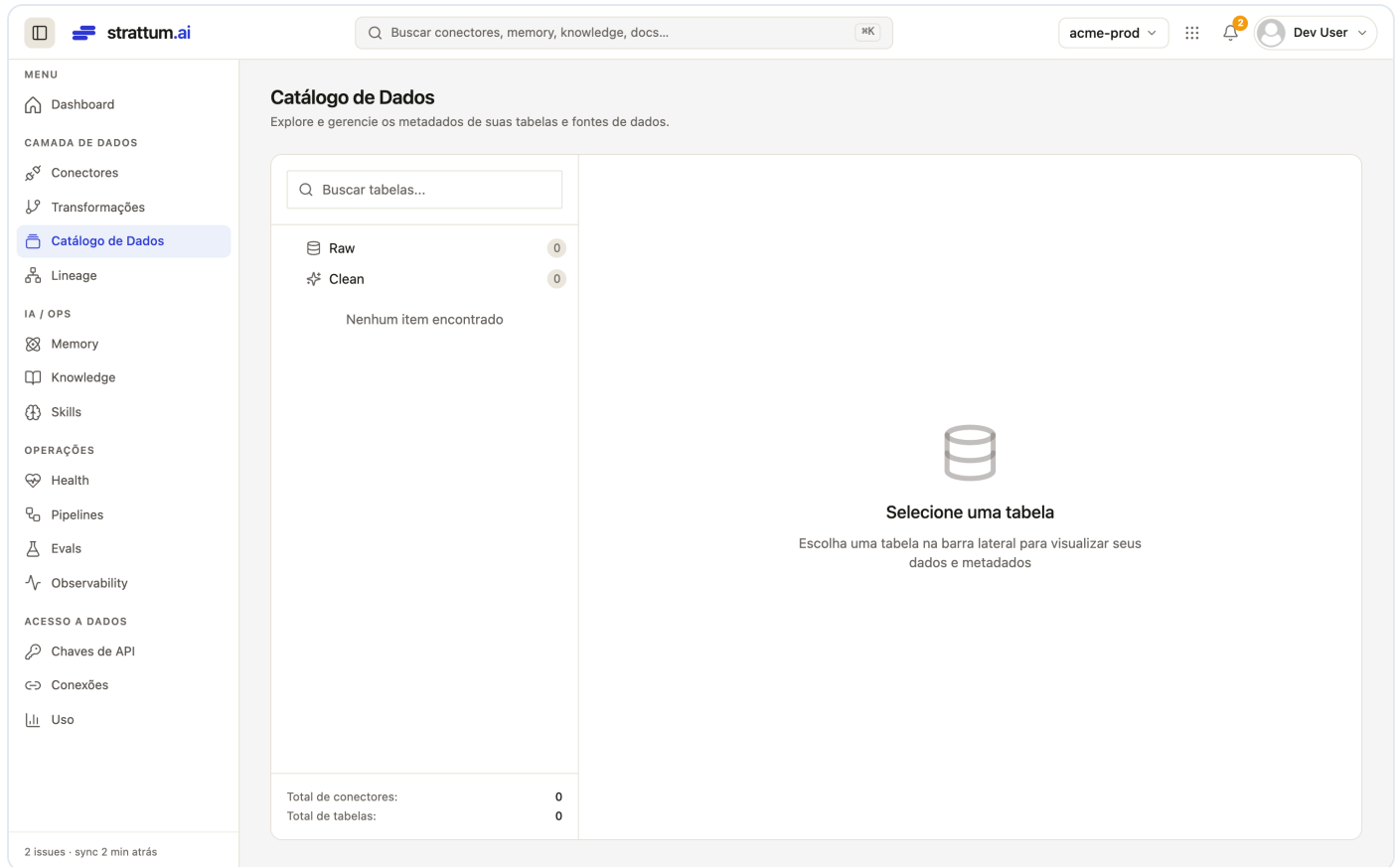
Tudo num único servidor **BYOC** (na nuvem do próprio cliente), sem data warehouse proprietário. **O que escala pra bilhões (lake, ingestão, incremental) é barato; o que é limitado por RAM (grafo) sobe com o servidor — e o custo permanente é sempre o do delta.**

Fontes de preço (jul/2026, on-demand Linux): tabela oficial AWS — `r8g.large` sa-east-1 US\$ 0,2216/h e `t4g.large` sa-east-1 US\$ 0,1072/h (⇒ `t4g.xlarge` US\$ 0,2144/h); us-east-1: `t4g.xlarge` US\$ 0,1344/h · `r8g.large` US\$ 0,1178/h; EBS gp3 sa-east-1 US\$ 0,152/GB-mês. Mensal = hora × 730. S3 Standard sa-east-1 US\$ 0,0405/GB-mês (primeiros 50 TB) — conferir na calculadora AWS ao fechar proposta; câmbio R\$ não aplicado.

Antes → Depois (console Strattum)

Toda a jornada é visível no console — sem SQL na mão.

Antes — catálogo vazio, memória vazia:



strattum.ai

Buscar conectores, memory, knowledge, docs...

acme-prod

Dev User

MENU

Dashboard

CAMADA DE DADOS

Conectores

Transformações

Catálogo de Dados

Lineage

IA / OPS

Memory

Knowledge

Skills

OPERAÇÕES

Health

Pipelines

Evals

Observability

ACESSO A DADOS

Chaves de API

Conexões

Uso

2 issues - sync 2 min atrás

Memory

Memória episódica de longo prazo da organização.

NER Review

Nenhuma entidade corresponde aos filtros atuais.

Depois — 3M linhas catalogadas + grafo materializado:

Catálogo: 3 tabelas `raw` + 5 `clean` = 8 no total (era 0), sem ninguém escrever SQL de ETL.

strattum.ai

Buscar conectores, memory, knowledge, docs...

acme-prod

Dev User

MENU

Dashboard

CAMADA DE DADOS

Conectores

Transformações

Catálogo de Dados

Lineage

IA / OPS

Memory

Knowledge

Skills

OPERAÇÕES

Health

Pipelines

Evals

Observability

ACESSO A DADOS

Chaves de API

Conexões

Uso

2 issues - sync 2 min atrás

Catálogo de Dados

Explore e gerencie os metadados de suas tabelas e fontes de dados.

Buscar tabelas...

Raw3

MongoDB2

PostgreSQL1

Clean5

Selezione uma tabela

Escolha uma tabela na barra lateral para visualizar seus dados e metadados

Total de conectores:2

Total de tabelas:8

Camada *clean* com as contagens reais — **users** 2.100.000, **contracts** 1.050.000, **products** 500, mais as fatias do grafo:

strattum.ai

Buscar conectores, memory, knowledge, docs...

acme-prod

Dev User

MENU

Dashboard

CAMADA DE DADOS

Conectores

Transformações

Catálogo de Dados

Lineage

IA / OPS

Memory

Knowledge

Skills

OPERAÇÕES

Health

Pipelines

Evals

Observability

ACESSO A DADOS

Chaves de API

Conexões

Uso

Catálogo de Dados

Explore e gerencie os metadados de suas tabelas e fontes de dados.

Buscar tabelas...

Raw

MongoDB

PostgreSQL

Clean

contracts

graph_contracts

graph_users

products

users

Total de conectores:

2

Total de tabelas:

8

users

8 campos 2.100.000 linhas

Dados Estrutura

Buscar nas linhas...

Arquivo: lake://clean.users — exibindo 50 de 2.100.000 linhas

USER_ID	EXTERNAL_ID	NAME	EMAIL	CPF	CITY
299	299	Usuário 299	user299@example.com	0000000299	Curitiba
599	599	Usuário 599	user599@example.com	0000000599	Fortalez
899	899	Usuário 899	user899@example.com	0000000899	Curitiba
1199	1199	Usuário 1199	user1199@example.com	0000001199	Fortalez
1499	1499	Usuário 1499	user1499@example.com	0000001499	Curitiba
1799	1799	Usuário 1799	user1799@example.com	0000001799	Fortalez
2099	2099	Usuário 2099	user2099@example.com	0000002099	Curitiba
2399	2399	Usuário 2399	user2399@example.com	0000002399	Fortalez
2699	2699	Usuário 2699	user2699@example.com	0000002699	Curitiba
2999	2999	Usuário 2999	user2999@example.com	0000002999	Fortalez
3299	3299	Usuário 3299	user3299@example.com	0000003299	Curitiba

Página 1 de 42000 · linhas 1–50 de 2.100.000

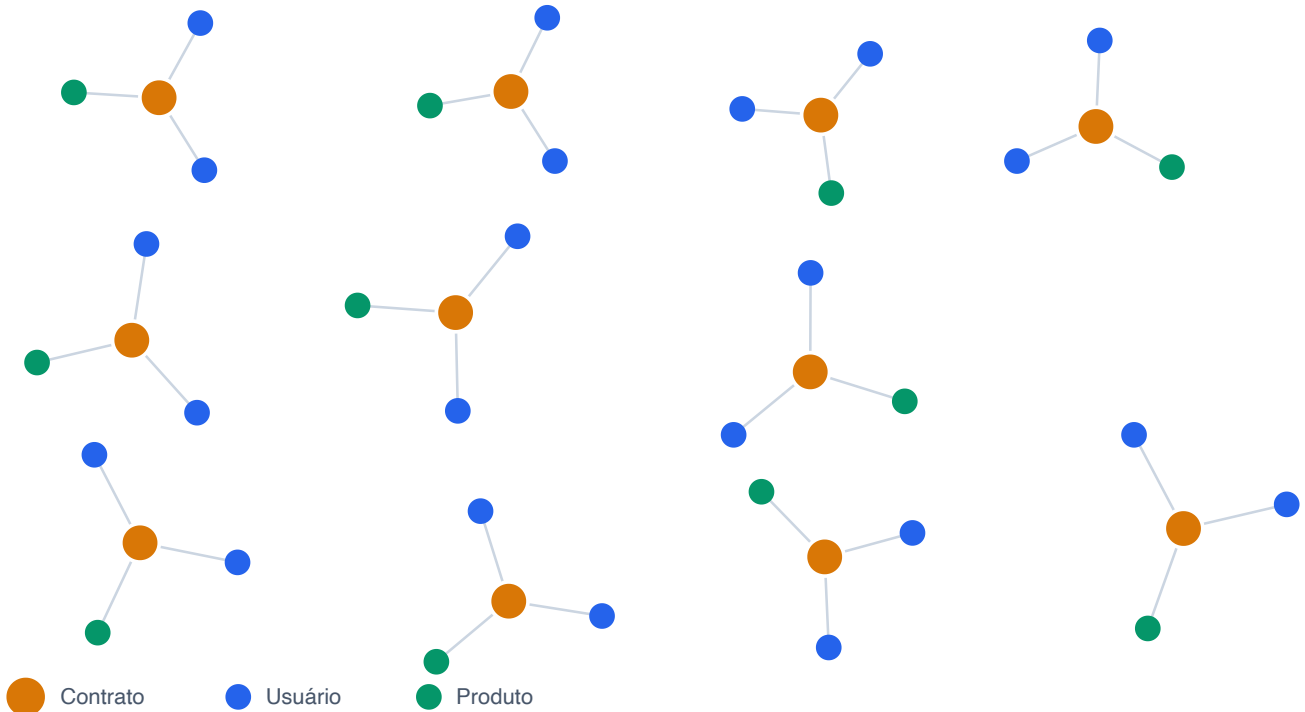
< Anterior

Próximo >

Grafo de entidades no Memory — o primeiro paint agora abre **conectado**: uma amostra semeada por arestas (~800 nós) traz os **contratos** (âmbar) ligando suas duas partes — os **usuários** (azul) — e o **produto** (verde). São os "digital twins" da §4, repetidos ~200x. O esquema do padrão:

Primeiro paint — agora conectado

Amostra semeada por arestas: cada contrato (●) liga suas duas partes (usuários) e o produto — os "digital twins", repetidos.



E o mesmo **ao vivo no console** — repare no rodapé: **800 entidades • 627 conexões** (antes: 0 conexões):

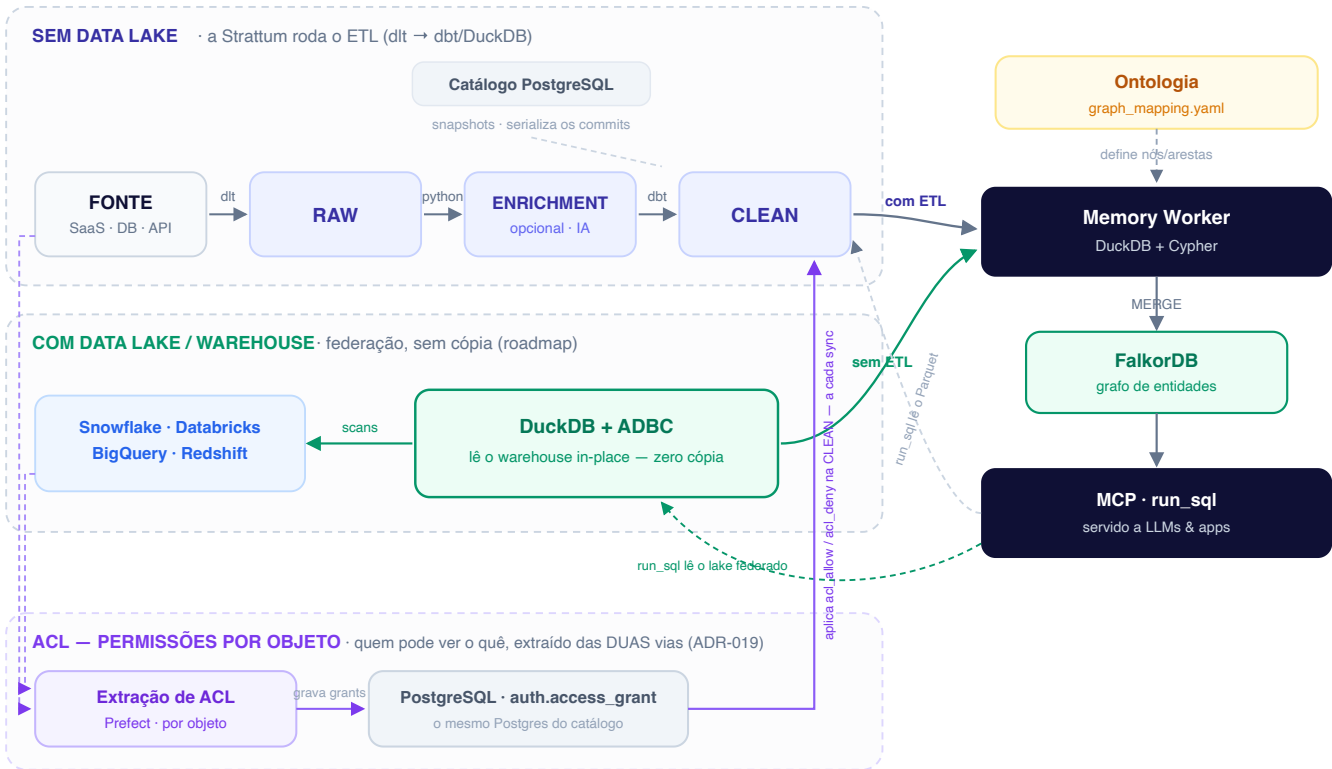
A captura de tela mostra a interface do console de gerenciamento de dados do Strattum.ai. No topo, há uma barra de busca com o texto "Buscar conectores, memory, knowledge, docs...". À direita, há uma barra de seleção de contexto com "acme-prod" e um ícone de notificação. O menu lateral à esquerda contém opções como "Dashboard", "Conectores", "Transformações", "Catálogo de Dados", "Lineage", "Memory" (destacado), "Knowledge", "Skills", "Operações", "Health", "Pipelines", "Evals", "Observability", "Acesso a Dados", "Chaves de API", "Conexões" e "Uso".

O painel principal, intitulado "Memory", apresenta uma visualização de rede densa com 800 entidades e 627 conexões. A interface inclui uma barra de busca para entidades, filtros para "Entidades" e "Views", e uma barra de status no rodapé que indica "800 entidades • 627 conexões". A visualização da rede mostra uma complexa interconexão de nós coloridos (laranja, azul e verde) representando as entidades e suas relações.

Como funciona (4 passos)

Como o dado entra, vira grafo e é servido

Dois caminhos de ingestão → um só grafo de conhecimento → servido por SQL e por MCP



1. **Conecte** a fonte no console (PostgreSQL, MongoDB, ...). As credenciais ficam no registro de conectores; a seleção de tabelas/coleções e o modo de sync (full/incremental, cursor, PK) ficam no estado do conector.
2. **Ingira** para o `raw` do lake — Parquet em S3/MinIO, catalogado pelo **DuckLake**. Streaming ponta a ponta: memória constante, incremental por cursor.
3. **Modele** com **dbt** rodando DuckDB nativo sobre o lake: normalização, `JOIN`s, tipagem → camada `clean`, materializada de volta no próprio lake.
4. **Construa o grafo**: a ontologia mapeia `clean` → nós e arestas; o worker resolve entidades e materializa no FalkorDB. O resultado aparece no Memory do console.

Tudo sobre **formatos abertos** (Parquet + catálogo relacional). Sem lock-in de warehouse, sem cópia proprietária dos dados.

Por que isso importa

- **Qualquer empresa consegue.** Se roda num notebook de 8 GB com 3M de linhas em ~1 minuto, roda no seu ambiente com os seus dados.
- **Barato em regime permanente.** O sync diário custa o **tamanho do delta**, não o da base.
- **Aberto por construção.** Parquet + DuckLake: os dados continuam seus, consultáveis por qualquer engine, sem reimportar.
- **Do dado bruto ao conhecimento.** Não para no "data lake": entrega camada modelada + grafo de entidades pronto para IA/contexto.

Metodologia & reprodutibilidade

- **Medição:** cada estágio rodou como processo isolado sob `/usr/bin/time -l` (tempo de parede + *maximum resident set size*). Throughput = linhas ÷ tempo do estágio.
- **Dados:** sintéticos e determinísticos (geradores versionados). `updated_at` do incremento estritamente maior que o watermark do bootstrap, para exercitar o delta real.
- **Código de produção:** a ingestão usa os conectores/flows reais da plataforma (`sync_resource_to_raw`), não um caminho de benchmark à parte. O `clean` usa os mesmos modelos dbt. O grafo usa o mesmo `MemoryWorkerPipeline`.
- **Honestidade de escopo:** ingestão e clean no volume cheio (2M/1M); grafo em fatia representativa com projeção, pela restrição de RAM do notebook (ver §4).
- **O benchmark virou QA:** de brinde, ele expôs (e este relatório corrigiu) o gargalo de escrita do grafo — 1 `MERGE` /linha → `UNWIND` batch, **2,87x mais rápido** — e o cap de first-paint do viz que estourava em grafos grandes.

Benchmark executado em 2026-07-19; grafo otimizado (UNWIND) no mesmo dia.